

Reviewer Pack — SOS Degree Lower Bound via Adjacency Eigenvalue Multiplicity

Entry 8131a890bdd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 06:28:29 UTC

SOS Degree Lower Bound via Adjacency Eigenvalue Multiplicity

Entry ID: 8131a890bdd5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 06:28:29 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Refutation Degree for Max-CUT

Statement:

For any n -vertex graph G , the SOS refutation degree $d(G)$ required to certify $\text{Max-CUT}(G) \leq \alpha$ satisfies $d(G) \geq k(G)$, where $k(G)$ is the number of distinct positive eigenvalues of G 's adjacency matrix. Equality holds for complete bipartite graphs.

Rationale (proposer's reasoning):

The Positivstellensatz provides certificates for non-negativity of polynomials, which underlie SOS proofs. The multiplicity of positive eigenvalues reflects the algebraic structure of the quadratic form defining Max-CUT, creating a bridge between spectral geometry and proof complexity.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c5d3783e87243034

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        n = len(B[0])
        result = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(n)] for i in range(len(A))]
        return result

    def transpose(matrix):
        return [list(row) for row in zip(*matrix)]

    def gaussian_elimination(A, b):
        n = len(A)
        augmented_matrix = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = max(range(i, n), key=lambda x: abs(augmented_matrix[x][i]))
            augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
            for j in range(i + 1, n):
                factor = augmented_matrix[j][i] / augmented_matrix[i][i]
                augmented_matrix[j] = [augmented_matrix[j][k] - factor * augmented_matrix[i][k] for k in range(i + 1, n)]
        x = [0] * n
        for i in range(n - 1, -1, -1):
            x[i] = (augmented_matrix[i][-1] - sum(augmented_matrix[i][j] * x[j] for j in range(i + 1, n))) / augmented_matrix[i][i]
        return x

    def solve_linear_system(A, b):
        x = gaussian_elimination(A, b)
        return x

    def eigenvalues(matrix):
        n = len(matrix)
        identity = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
        A = matrix
        B = identity
        tol = 1e-6
        max_iter = 1000
```

```

    for _ in range(max_iter):
        Q, R = qr_decomposition(A)
        A = matrix_multiply(R, Q)
        if max(abs(A[i][j]) for i in range(n) for j in range(i + 1)) < tol:
            break
    eigenvals = [A[i][i] for i in range(n)]
    return eigenvals

def qr_decomposition(matrix):
    n = len(matrix)
    Q = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    R = matrix.copy()
    for k in range(n):
        v = [R[i][k] for i in range(k, n)]
        norm_v = math.sqrt(sum(x**2 for x in v))
        e_k = [0 if i != k else 1 / norm_v for i in range(n)]
        Q[k] = [v[i] - norm_v * e_k[i] for i in range(n)]
        R = [[sum(Q[j][i] * R[j][k] for j in range(k, n)) for k in range(n)] for i in range(n)]
    return Q, R

def degree_optimized_sdp(A):
    n = len(A)
    identity = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    A_bar = matrix_multiply(transpose(A), A)
    b = [sum(A[i][j] * (i + j) for j in range(i, n)) / 2 for i in range(n)]
    x = solve_linear_system(A_bar, b)
    return sum(x[i]**2 for i in range(n))

n = 40
A = [[random.choice([0, 1]) if i != j else 0 for j in range(n)] for i in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        if random.random() < 0.5:
            A[i][j] = A[j][i] = 1

eigenvals = eigenvalues(A)
k_G = sum(1 for val in set(eigenvals) if val > 0)

d_G = degree_optimized_sdp(A)

return {
    "metric_name": "SOS Degree",
    "metric_value": d_G,
    "instances_tested": 1,
    "conjecture_holds": d_G >= k_G,
    "counterexample": "" if d_G >= k_G else f"Graph with {n} vertices, k(G)={k_G}, d(G)={d_G}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(res["metric_value"] for res in results) / len(results)
std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4a38.py", line 108, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4a38.py", line 89, in run_trial
    eigenvals = eigenvalues(A)
                  ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4a38.py", line 55, in eigenvalues
    Q, R = qr_decomposition(A)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4a38.py", line 70, in qr_decomposition
    Q[k] = [v[i] - norm_v * e_k[i] for i in range(n)]
            ~^^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during eigenvalue computation, preventing validation of conjecture | next:
Fix the QR decomposition implementation to handle edge cases in eigenvalue calculation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	115635
2	propose	ollama_remote	qwen3:8b	0	0	107954
3	preregistration	ollama_remote	qwen3:8b	0	0	24662
4	novelty	ollama_remote	qwen3:8b	0	0	20688
5	novelty	ollama_remote	qwen3:8b	0	0	12497
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16519
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14290
8	judge	ollama_remote	qwen3:8b	0	0	16013

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 328259 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/8131a890bdd5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8131a890bdd5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8131a890bdd5.tar.gz` (if generated)